Exercise 6 – Shapley Values

Interpretable Machine Learning

Hint: Useful libraries

R

```r
library(sets)       # power sets (Exercise 1)
library(combinat)   # permutations (Exercise 1)
library(ranger)     # random forest (Exercise 3)
library(randomForest)  # random forest for the LIME explainer (Exercise 4)
library(DALEX)      # model explainers (Exercise 4)
library(DALEXtra)   # LIME interface (Exercise 4)

set.seed(123456)
```

Python

```python
import itertools
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.special import comb
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import lime
import lime.lime_tabular

np.random.seed(123456)
```

Exercise 1: The Shapley value

Learning goals

1. Compute payoffs of coalitions in a cooperative game.
2. Implement the exact Shapley value via both the set and the permutation definition.
3. Implement the permutation-based approximation of Shapley values.
4. Check the Shapley axioms empirically for a concrete game.

In this exercise, we implement the original Shapley value for a cooperative game. The game we consider here is an AI quiz, where we have five players: Timnit, Margret, Samy, Jeff, and Larry. The overall payoff function for a set of players S is given as

$$\text{payoff}(S) = 10t + 10m + 10s + 2j + 20(t \wedge m) + 20(t \wedge m \wedge s) - 30((t \vee m \vee s) \wedge j)$$

where t, m, s, j, l indicate whether Timnit, Margret, Samy, Jeff, and Larry are in the set S. The function `payoff(coalition)` is supposed to implement this functionality, where a set is a list/set of the respective characters (`'t', 's', 'm', 'j', 'l'`).

a) Calculate the payoff of $S =$ {`t,m`} (result should be 40) and $S =$ {`t,j,s`} (result should be -8).

Solution

$$\text{payoff}(\{\texttt{t,m}\}) = 10 + 10 + 20 = 40$$
$$\text{payoff}(\{\texttt{t,j,s}\}) = 10 + 10 + 2 - 30 = -8$$

b) Implement the original, exact Shapley value algorithm, using the set definition of Shapley values:

- Start by implementing the function `payoff(coalition)`, which should return the payoff of a subset `coalition` as given in the task. You can do this, for example, by first assigning a Boolean value for each player.
- Define a function `all_unique_subsets(population)` returning all subsets of a given set `population`.
- Finally, implement `shapley_set(member, population, v_function)` calculating the Shapley value for an arbitrary value function using the set definition.

Test your functions with the examples from a) or with other examples from this game, i.p. corner cases like the empty set or the full set of all players.

Solution

Pseudocode of `payoff(coalition)`:

```
Require coalition: coalition vector (set or list or ...)
  t <- boolean if 't' is in coalition  (same for s, m, j, l)
  return 10*t + 10*m + 10*s + 2*j + 20*(t and m)
         + 20*(t and m and s) - 30*((t or m or s) and j)
```

For the corner cases we get payoff($\emptyset$) $= 0$ and payoff($\{$`t,m,s,j,l`$\}$) $= 10 + 10 + 10 + 2 + 20 + 20 - 30 = 42$.

R

```r
### Define the payoff function.
payoff = function(coalition) {
  ### Define boolean variables that indicate whether Timnit, Margret, Samy, Jeff
  ### and Larry are in the set or not.
  t = 't' %in% coalition
  s = 's' %in% coalition
  m = 'm' %in% coalition
  j = 'j' %in% coalition
  l = 'l' %in% coalition

  ### Definition of the payoff as given in the task.
  10*t + 10*m + 10*s + 2*j + 20*(t & m) + 20*(t & m & s) - 30*((t | m | s) & j)
}

### Testing, including the corner cases empty set and full population.
population = c('j', 'l', 'm', 's', 't')
payoff(c('t', 'm'))
payoff(c('t', 'j', 's'))
payoff(c())
payoff(population)
```

40

-8

0

42

Python

```python
def payoff(coalition):
    # Define boolean variables that indicate whether Timnit, Margret, Samy,
    # Jeff and Larry are in the set or not.
    t = 't' in coalition
    s = 's' in coalition
    m = 'm' in coalition
    j = 'j' in coalition
    l = 'l' in coalition
    return 10*t + 10*m + 10*s + 2*j + 20 * (t and m) + 20 * (t and m and s) - 30 * ((t or m or s) ar


# Testing, including the corner cases empty set and full population.
population = ['j', 'l', 'm', 's', 't']
print(payoff(('t', 'm')))
print(payoff(['t', 'j', 's']))
print(payoff([]))
print(payoff(population))
```

```
40
-8
0
42
```

Concerning `all_unique_subsets(population)`, both R and Python provide built-in functions that return the power set, i.e. the set of all subsets of a given set. Alternatively, the power set can be built recursively: pick an element, compute the power set of the remaining elements, and return those subsets once unchanged and once with the picked element added.

R

```r
all_unique_subsets = function(population) {
  ### Use the built-in set_power function and convert to a list for further processing.
  as.list(set_power(population))
}

### Testing
print(all_unique_subsets(c()))
print(all_unique_subsets(c(2, 4)))
print(all_unique_subsets(population))
```

```
[[1]]
{}

[[1]]
{}

[[2]]
{2}

[[3]]
{4}

[[4]]
{2, 4}

[[1]]
{}

[[2]]
{"j"}

[[3]]
{"l"}

[[4]]
{"m"}

[[5]]
{"s"}

[[6]]
{"t"}

[[7]]
{"j", "l"}

[[8]]
{"j", "m"}

[[9]]
{"j", "s"}
```

```
[[10]]
{"j", "t"}

[[11]]
{"l", "m"}

[[12]]
{"l", "s"}

[[13]]
{"l", "t"}

[[14]]
{"m", "s"}

[[15]]
{"m", "t"}

[[16]]
{"s", "t"}

[[17]]
{"j", "l", "m"}

[[18]]
{"j", "l", "s"}

[[19]]
{"j", "l", "t"}

[[20]]
{"j", "m", "s"}

[[21]]
{"j", "m", "t"}

[[22]]
{"j", "s", "t"}

[[23]]
{"l", "m", "s"}
```

```
[[24]]
{"l", "m", "t"}

[[25]]
{"l", "s", "t"}

[[26]]
{"m", "s", "t"}

[[27]]
{"j", "l", "m", "s"}

[[28]]
{"j", "l", "m", "t"}

[[29]]
{"j", "l", "s", "t"}

[[30]]
{"j", "m", "s", "t"}

[[31]]
{"l", "m", "s", "t"}

[[32]]
{"j", "l", "m", "s", "t"}
```

Python

```python
def all_unique_subsets(population):
    """Recursive power set: pick an element, compute the power set of the
    rest, return those subsets with and without the picked element."""
    if not population:
        return [[]]
    else:
        population = list(set(population))  # remove double elements
        subsets_wo_member = all_unique_subsets(population[1:])
        subsets_w_member = [s + [population[0]] for s in subsets_wo_member]
        return subsets_w_member + subsets_wo_member

# Alternative with the built-in powerset recipe:
# https://docs.python.org/3/library/itertools.html#itertools-recipes
# def all_unique_subsets(population):
#     population = list(set(population))
#     power_set = itertools.chain.from_iterable(
#         itertools.combinations(population, r) for r in range(len(population) + 1))
#     return [list(subset) for subset in power_set]


# Testing
print(all_unique_subsets([]))
print(all_unique_subsets(['t']))
print(all_unique_subsets(population))
```

```
[[]]
[['t'], []]
[['j', 'l', 'm', 't', 's'], ['l', 'm', 't', 's'], ['j', 'm', 't', 's'], ['m', 't', 's']
```

For the Shapley value itself we follow the set definition. Pseudocode of `shapley_set(member, population, v_function)`:

```
Require member: individual player, or feature of interest
Require population: vector containing all available players
Require v_function: some value function
  remainder          <- population \ member
  all_sets_wo_member <- all_unique_subsets(remainder)
  F      <- length of population
  result <- 0
  for coalition in all_sets_wo_member:
      S      <- length of coalition
      diff   <- v_function(coalition + member) - v_function(coalition)
```

```
      factor <- S! * (F - S - 1)! / F!
      result <- result + factor * diff
  return result
```

We first define a few helper functions that are reused throughout this exercise (payoff differences for a given list of coalitions, all coalitions without given members, and a test function that also checks the efficiency axiom):

R

```r
### A function which returns a list of all possible coalitions (represented as
### sets) without (one or more) specific members.
all_coalitions_without_members = function(members, population) {
  population_without_members = setdiff(population, members)
  as.list(all_unique_subsets(population_without_members))
}

### The payoff difference of adding a member to one coalition / to a list of
### coalitions. The v_function argument makes these reusable for arbitrary games.
payoff_diff = function(member, coalition, v_function = payoff) {
  v_function(c(member, coalition)) - v_function(coalition)
}

payoff_diff_list = function(member, coalitions, v_function = payoff) {
  sapply(1:length(coalitions), function(i) payoff_diff(member, coalitions[[i]], v_function))
}

### All Shapley values of a population, and a test function that prints them
### and also compares their sum to the total payoff (efficiency axiom).
all_shapley_values = function(population, shapley_fct, v_function = payoff) {
  sapply(1:length(population), function(i) shapley_fct(population[i], population, v_function))
}

shapley_test = function(population, shapley_fct, v_function = payoff) {
  shapley_values = all_shapley_values(population, shapley_fct, v_function)
  print(shapley_values)
  sum_ = sum(shapley_values)
  print(sum_)
  print(v_function(population))
  print(sum_ == v_function(population))
}
```

Python

```
def all_coalitions_without_members(members, population):
    population_without_members = [p for p in population if p not in members]
    return all_unique_subsets(population_without_members)


# The payoff difference of adding a member to one coalition / to a list of
# coalitions. The extra *args make these reusable for arbitrary value functions.
def payoff_diff(member, coalition, v_function=payoff, *args):
    return v_function(coalition + [member], *args) - v_function(coalition, *args)


def payoff_diff_list(member, coalitions, v_function=payoff, *args):
    return np.asarray([payoff_diff(member, coalition, v_function, *args) for coalition in coalitions


# All Shapley values of a population, and a test function that prints them
# and also compares their sum to the total payoff (efficiency axiom).
def all_shapley_values(population, shapley_fct, v_function=payoff, *args):
    return {member: shapley_fct(member, population, v_function, *args) for member in population}


def shapley_test(population, shapley_fct, v_function=payoff, *args):
    shapley_values = all_shapley_values(population, shapley_fct, v_function, *args)
    print("\n".join(f"Player {player}: {shapley_values[player]}" for player in shapley_values))
    sum_ = sum(shapley_values.values())
    print(sum_)
    print(v_function(population, *args))
    print(math.isclose(sum_, v_function(population, *args)))  # efficiency axiom
```

With these, the set definition is a weighted sum of payoff differences:

R

```
### Calculate the Shapley value for a player using the set definition.
shapley_set = function(member, population, v_function = payoff) {
  population = population[!duplicated(population)] # remove double elements
  all_coalitions_without_member = all_coalitions_without_members(member, population)

  ### Compute the payoff differences with and without the member player for
  ### all coalitions at once.
  payoff_diffs = payoff_diff_list(member, all_coalitions_without_member, v_function)

  ### Calculate the weights, which are the combinatorical number of
  ### occurrences of each set.
  F = length(population)
  weights = sapply(1:length(all_coalitions_without_member), function(i) {
    cardinality_coalition = length(all_coalitions_without_member[[i]])
    factorial(cardinality_coalition) * factorial(F - cardinality_coalition - 1) / factorial(F)
  })

  ### The Shapley value now is the weighted sum of the differences.
  sum(weights * payoff_diffs)
}

### Testing and sanity check (the last line checks the efficiency axiom).
print(shapley_set("t", population))
shapley_test(population, shapley_set)
```

```
[1] 24.16667
[1] -20.50000   0.00000  24.16667  14.16667  24.16667
[1] 42
[1] 42
[1] TRUE
```

Python

```
def shapley_set(member, population, v_function=payoff, *args):
    """Shapley value of `member` via the set definition."""
    population = list(set(population))  # remove double elements
    all_coalitions_without_member = all_coalitions_without_members([member], population)

    # Payoff differences with and without the member for all coalitions.
    payoff_diffs = payoff_diff_list(member, all_coalitions_without_member, v_function, *args)

    # The weights are the combinatorical number of occurrences of each set.
    F = len(population)
    weights = np.asarray([math.factorial(len(coalition)) * math.factorial(F - len(coalition) - 1) /
                          for coalition in all_coalitions_without_member])

    # The Shapley value is the weighted sum of the differences.
    return np.sum(weights * payoff_diffs)


# Testing and sanity check (the last line checks the efficiency axiom).
print(shapley_set('t', population))
shapley_test(population, shapley_set)
```

```
24.166666666666668
Player j: -20.5
Player l: 0.0
Player m: 24.166666666666668
Player s: 14.166666666666668
Player t: 24.166666666666668
42.0
42
True
```

c) Analogously, implement the exact Shapley value algorithm using the order definition (or permutation definition) `shapley_perm(member, population, v_function)`. For this purpose, you can write (or use) a function that returns all orderings / all permutations of a given set. Again, test your function as in b).

Solution

A function to generate all permutations can be implemented similar to the function `all_unique_subsets()`, also by first generating the first element and then calling the function on the set of remaining elements; both languages also provide built-in functions for this.

Pseudocode of `shapley_perm(member, population, v_function)`:

```
Require member, population, v_function (as before)
  all_perms <- all permutations of population
  F         <- length of population
  result <- 0
  for perm in all_perms:
      member_ix <- index of member in perm
      coalition <- players of perm before member_ix
      diff      <- v_function(coalition + member) - v_function(coalition)
      result    <- result + diff
  return result / F!
```

R

```r
### Calculate a Shapley value using the permutation-based definition.
shapley_perm = function(member, population, v_function = payoff) {
  ### Generate all permutations from the population (as list of vectors).
  all_perms = combinat::permn(population)

  ### Find the index of the member player for every permutation.
  member_idxs = sapply(all_perms, function(x) match(member, x))

  ### Get the coalition for each permutation (which are just all the players
  ### with an index lower than the member player).
  coalitions = lapply(1:factorial(length(population)), function(i) all_perms[[i]][1:member_idxs[i] -

  ### The Shapley value is the mean of all the payoff differences.
  differences = payoff_diff_list(member, coalitions, v_function)
  mean(differences)
}

### Testing
shapley_test(population, shapley_perm)
```

```
[1] -20.50000   0.00000  24.16667  14.16667  24.16667
[1] 42
[1] 42
[1] TRUE
```

Python

```python
def shapley_perm(member, population, v_function=payoff, *args):
    """Shapley value of `member` via the permutation (order) definition."""
    # permutation[:permutation.index(member)] is exactly the coalition that
    # arrived before the member in this permutation.
    coalitions = [permutation[:permutation.index(member)] for permutation in itertools.permutations(
    return np.mean([payoff_diff(member, list(coalition), v_function, *args) for coalition in coaliti


# Testing and sanity check
shapley_test(population, shapley_perm)
```

```
Player j: -20.5
Player l: 0.0
Player m: 24.166666666666668
Player s: 14.166666666666666
Player t: 24.166666666666668
42.0
42
True
```

d) Next, implement the permutation-based approximation discussed in the lecture: `shapley_perm_approx(member, population, v_function, num_iter)`. Here, `num_iter` is a fixed number of iterations given.

Note that because we implement the Shapley values for game theory, not yet for machine learning, we are not concerned with PD-functions and with data sets, but we can calculate the payoff function directly. This means that sampling is only required concerning the permutations, the sampling concerning the data points can be omitted.

Again, don't forget to test your function. Compare it to the function in c) concerning runtime and accuracy for different numbers of iterations.

Solution

Pseudocode of `shapley_perm_approx(member, population, v_function, num_iter)`:

```
Require member, population, v_function (as before)
Require num_iter: number of iterations
  vals <- empty vector or list
  for i in num_iter:
      perm      <- draw a random permutation of population
      member_ix <- index of member in perm
```

```
      coalition <- players of perm before member_ix
      vals[i]   <- v_function(coalition + member) - v_function(coalition)
  return average of vals
```

R

```r
### Calculate the permutation-based approximation of the Shapley value with
### num_iter iterations.
shapley_perm_approx = function(member, population, v_function = payoff, num_iter = 100) {
  ### Sample num_iter permutations (the single columns are the permutations).
  perms = replicate(num_iter, sample(population))

  ### Find the index of the member player in every permutation (column).
  member_idxs = apply(perms, 2, function(x) match(member, x))

  ### Get the coalition for each permutation (all players before the member).
  coalitions = lapply(1:num_iter, function(i) perms[1:member_idxs[i] - 1, i])

  ### The estimate is the mean of the payoff differences over all sampled
  ### coalitions, same as for the exact order definition.
  differences = payoff_diff_list(member, coalitions, v_function)
  mean(differences)
}

### Testing: convergence for a growing number of iterations.
for (i in 2:4) {
  print(shapley_perm_approx('t', population, num_iter = 10^i))
}
shapley_test(population, shapley_perm_approx)
```

```
[1] 23.6
[1] 25.09
[1] 24.64
[1] -21.4   0.0  26.3  13.1  24.5
[1] 42.5
[1] 42
[1] FALSE
```

Python

```python
def shapley_perm_approx(member, population, v_function=payoff, *args, its=100):
    """Permutation-based approximation with a fixed number of iterations."""
    vals = []
    rng = np.random.default_rng()
    for i in range(its):
        permutation = list(rng.permutation(population))
        coalition = permutation[:permutation.index(member)]
        vals.append(payoff_diff(member, coalition, v_function, *args))
    return np.mean(vals)


# Testing: convergence for a growing number of iterations.
for its in [100, 1000, 10000]:
    print(shapley_perm_approx('t', population, its=its))
shapley_test(population, shapley_perm_approx)
```

```
22.0
25.32
24.155
Player j: -20.2
Player l: 0.0
Player m: 25.6
Player s: 12.0
Player t: 21.2
38.6
42
False
```

Clear convergence is apparent with a growing number of iterations; the exact numbers depend on the specific parameters and the seed.

e) In the last part of the exercise, we want to test whether the Shapley values or specific sets fulfill the axioms that uniquely define the Shapley values. Please write the functions `efficiency_check`, `symmetry_check`, `dummy_check`, and `additivity_check` to check the respective properties, all of which return a Boolean value. We briefly recall the properties below:

(i) *Efficiency:* Player contributions add up to the total payout of the game:

$$\sum_{j=1}^{p} \phi_j = v(P).$$

(ii) *Symmetry:* Assess whether two features with the same contributions have the same Shapley value. I.e., for any two players $\{j, k\}$, if for any $S \subseteq P \setminus \{j, k\}$ we have

$$v(S \cup \{j\}) - v(S) = v(S \cup \{k\}) - v(S),$$

then the Shapley values of j and k should be identical.

(iii) *Dummy:* Given a player j which makes no contribution for all subsets $S \subseteq P \setminus \{j\}$ (that is $v(S) = v(S \cup \{j\})$), the Shapley value of this player must be zero.

(iv) *Additivity:* If v is the sum of two games v_1 and v_2 (i.e. $v(S) = v_1(S) + v_2(S)$ for any S), then the payout is the sum of payouts: $\phi_{j,v} = \phi_{j,v_1} + \phi_{j,v_2}$.

Again, test these functions and use them to check whether the axioms are fulfilled for our specific example.

Solution

Each check follows the same pattern: compute the quantities on both sides of the axiom and compare them. For symmetry and dummy, the premise of the axiom is checked first; if the premise does not hold for the given players, the axiom holds vacuously and the check returns true.

R

```r
### Symmetry: if the premise (equal payoff differences for all coalitions
### without j and k) holds, the Shapley values of j and k must be identical;
### if the premise is violated, the axiom holds vacuously.
symmetry_check = function(j, k, population) {
  all_coalitions_without_candidates = all_coalitions_without_members(c(j, k), population)
  payoff_diff_j = payoff_diff_list(j, all_coalitions_without_candidates)
  payoff_diff_k = payoff_diff_list(k, all_coalitions_without_candidates)
  all(any(payoff_diff_j != payoff_diff_k) | shapley_set(j, population) == shapley_set(k, population)
}

### Check the symmetry property for all possible combinations of two players.
all_combinations_two_players = as.list(set_combn(population, m = 2))
symmetry_check_all_combinations = sapply(1:length(all_combinations_two_players),
  function(i) symmetry_check(as.list(all_combinations_two_players[[i]])[1],
                             as.list(all_combinations_two_players[[i]])[2],
                             population))
print(all(symmetry_check_all_combinations))

### Dummy: a player whose payoff differences are all 0 must have Shapley value 0.
dummy_check = function(j, population) {
  all_coalitions_without_candidate = all_coalitions_without_members(j, population)
  payoff_diff_j = payoff_diff_list(j, all_coalitions_without_candidate)
  all(any(payoff_diff_j != 0) | shapley_set(j, population) == 0)
}

print(all(sapply(1:length(population), function(i) dummy_check(population[i], population))))

### Additivity: the Shapley values of the sum of two games are the sums of
### the games' Shapley values (rounding guards against numerical noise).
additivity_check = function(population, payoff_func_1, payoff_func_2) {
  payoff_func_1_and_2 = function(x) payoff_func_1(x) + payoff_func_2(x)
  all_shapley_values_payoff_func_1 = all_shapley_values(population, shapley_set, payoff_func_1)
  all_shapley_values_payoff_func_2 = all_shapley_values(population, shapley_set, payoff_func_2)
  all_shapley_values_payoff_func_1_and_2 = all_shapley_values(population, shapley_set, payoff_func_1
  all(round(all_shapley_values_payoff_func_1_and_2, 5)
    == round(all_shapley_values_payoff_func_1 + all_shapley_values_payoff_func_2, 5))
}

### Test with a decomposition of the original payoff into its first three and
### its remaining summands.
payoff_func_1 = function(coalition) {
  t = 't' %in% coalition; s = 's' %in% coalition; m = 'm' %in% coalition
  j = 'j' %in% coalition; l = 'l' %in% coalition
  10*t + 10*m + 10*s + 2*j
}
payoff_func_2 = function(coalition) {
  t = 't' %in% coalition; s = 's' %in% coalition; m = 'm' %in% coalition
  j = 'j' %in% coalition; l = 'l' %in% coalition
  20*(t & m) + 20*(t & m & s) - 30*((t | m | s) & j)
}
print(additivity_check(population, payoff_func_1, payoff_func_2))

### Efficiency: the Shapley values must sum to the payoff of the full population.
efficiency_check = function(population, v_function = payoff) {
  v_function(population) == sum(all_shapley_values(population, shapley_set, v_function))
}
print(efficiency_check(population))
```

```
[1] TRUE
[1] TRUE
[1] TRUE
[1] TRUE
```

Python

```
def symmetry_check(j, k, population, shapley_func, v_function, *args):
    # If the premise (equal payoff differences for all coalitions without j
    # and k) holds, the Shapley values of j and k must be identical; if it is
    # violated, the axiom holds vacuously.
    all_S = all_coalitions_without_members([j, k], population)
    surpluss_j = payoff_diff_list(j, all_S, v_function, *args)
    surpluss_k = payoff_diff_list(k, all_S, v_function, *args)
    if np.all(surpluss_j == surpluss_k):
        val_j = shapley_func(j, population, v_function, *args)
        val_k = shapley_func(k, population, v_function, *args)
        return math.isclose(val_j, val_k)
    return True


print(all(symmetry_check(j, k, population, shapley_set, payoff)
          for j, k in itertools.combinations(population, 2)))


def dummy_check(j, population, shapley_func, v_function, *args):
    # A player whose payoff differences are all 0 must have Shapley value 0.
    all_S = all_coalitions_without_members([j], population)
    surpluss_j = payoff_diff_list(j, all_S, v_function, *args)
    if np.sum(np.abs(surpluss_j)) == 0:
        return shapley_func(j, population, v_function, *args) == 0
    return True


print(all(dummy_check(j, population, shapley_set, payoff) for j in population))


def additivity_check(population, shapley_func, v_function1, v_function2, *args):
    # The Shapley values of the sum of two games are the sums of the games'
    # Shapley values (rounding guards against numerical noise).
    combined = lambda x: v_function1(x, *args) + v_function2(x, *args)
    vals1 = np.array([shapley_func(j, population, v_function1, *args) for j in population])
    vals2 = np.array([shapley_func(j, population, v_function2, *args) for j in population])
    vals_comb = np.array([shapley_func(j, population, combined) for j in population])
    return np.all(np.round(vals_comb, 5) == np.round(vals1 + vals2, 5))


# Test with a decomposition of the original payoff into its first three and
# its remaining summands.
def payoff_func_1(coalition):
    t, s, m, j = ('t' in coalition, 's' in coalition, 'm' in coalition, 'j' in coalition)
    return 10*t + 10*m + 10*s + 2*j


def payoff_func_2(coalition):
    t, s, m, j = ('t' in coalition, 's' in coalition, 'm' in coalition, 'j' in coalition)
    return 20 * (t and m) + 20 * (t and m and s) - 30 * ((t or m or s) and j)


print(additivity_check(population, shapley_set, payoff_func_1, payoff_func_2))


def efficiency_check(population, shapley_func, v_function, *args):
    # The Shapley values must sum to the payoff of the full population.
    payoff_total = v_function(population, *args)
    shapley_vals = all_shapley_values(population, shapley_func, v_function, *args)
    return math.isclose(payoff_total, sum(shapley_vals.values()))


print(efficiency_check(population, shapley_set, payoff))
```

```
True
True
True
True
```

All four axioms hold for our toy example (for the additivity check, we split the payoff function into two games that sum to the original one).

Exercise 2: Axioms of Shapley Values

Learning goals

1. Prove the symmetry, dummy and additivity axioms of Shapley values.
2. Explain the relationship between the set and the permutation definition of Shapley values.
3. Apply proof techniques such as index transformations and telescope sums to the Shapley formula.

In this exercise, we want to mathematically prove that the Shapley values in cooperative game theory, as defined in the lecture, fulfill the four axioms discussed in the lecture. Throughout this exercise, we assume that the order and the set definition of Shapley values yield the same result, so you can always choose which definition to use.

a) Prove that the symmetry, dummy player, and additivity axioms of Shapley values hold for any value function and any set P of players. These axioms are rather straightforward to prove, and this can be done using either of the definitions.

Solution

Setup. Let $P = \{1, \dots, p\}$ be the set of $p := |P|$ players, v the value function and ϕ_j the Shapley value of player j.

Recall the two equivalent definitions of Shapley values:

Permutation-based definition. Let $\text{Pred}_\pi(j)$ be the set of players that appear before player j in the permutation π. Then for any j, the Shapley value of player j is

$$\phi_j = \frac{1}{p!} \sum_{\pi \in \mathfrak{S}_P} \Big(v(\text{Pred}_\pi(j) \cup \{j\}) - v(\text{Pred}_\pi(j)) \Big),$$

where $\mathfrak{S}_P$ is the set of all $p!$ permutations of P. In words, we look at the "marginal contribution" of j each time it arrives in a permutation (depending on whichever players arrived before it), and then average over all permutations.

Set-based definition. For any j, we consider all subsets S of P which do not contain j, then the Shapley value of player j is

$$\phi_j = \sum_{S \subseteq P \setminus \{j\}} \frac{|S|!\,(p - |S| - 1)!}{p!} \Big(v(S \cup \{j\}) - v(S) \Big).$$

In essence, both definitions are almost the same, just that those terms from the first definition that yield the same subsets S (although from different permutations), are summed up and counted in the second definition, so that the second definition has less terms in the sum.

The dummy and additivity properties are relatively easy to prove.

Theorem (Null Player / Dummy). *Let $\{\phi_j\}_{j \in P}$ be the Shapley values induced by v. For any player $j \in P$ it holds: if $v(S \cup \{j\}) = v(S)$ for all $S \subseteq P \setminus \{j\}$, then $\phi_j = 0$.*

Proof. Assume $p \geq 1$, $j \in P$ and $v(S \cup \{j\}) = v(S)$ for all subsets $S \subseteq P \setminus \{j\}$. We need to prove that $\phi_j = 0$.

This follows directly from either of the two definitions: The assumption states that the difference $v(S \cup \{j\}) - v(S)$ inside the sum is 0, for every $S \subseteq P \setminus \{j\}$, in other words for every summand. Therefore, also the whole sum is 0, so $\phi_j = 0$. □

Theorem (Additivity). *Let $v, v_1, v_2 : 2^P \to \mathbb{R}$ be value functions with $v = v_1 + v_2$, which means that for every subset $S \subseteq P$ we have $v(S) = v_1(S) + v_2(S)$. Let $\{\phi_{j,v}\}_{j \in P}$ be the Shapley values induced by v and $\{\phi_{j,v_1}\}_{j \in P}$ and $\{\phi_{j,v_2}\}_{j \in P}$ those for v_1 and v_2 respectively. Then for any player $j \in P$ it holds:*

$$\phi_{j,v} = \phi_{j,v_1+v_2} = \phi_{j,v_1} + \phi_{j,v_2}.$$

Proof. Assume $p \geq 1$, $j \in P$ and that v_1, v_2 are two value functions. We need to prove that $\phi_{j,v_1+v_2} = \phi_{j,v_1} + \phi_{j,v_2}$.

For the marginal contributions of player j on any subset $S \subseteq P$ we have: $v(S \cup \{j\}) - v(S) = v_1(S \cup \{j\}) - v_1(S) + v_2(S \cup \{j\}) - v_2(S)$. Plugging this into either of the two definitions yields the result:

$$\phi_{j,v} = \phi_{j,v_1+v_2} = \sum_{S\subseteq P\setminus\{j\}} \frac{|S|!\,(p-|S|-1)!}{p!}\Big(v(S\cup\{j\})-v(S)\Big)$$
$$= \sum_{S\subseteq P\setminus\{j\}} \frac{|S|!\,(p-|S|-1)!}{p!}\Big(v_1(S\cup\{j\})-v_1(S)+v_2(S\cup\{j\})-v_2(S)\Big)$$
$$= \sum_{S\subseteq P\setminus\{j\}} \frac{|S|!\,(p-|S|-1)!}{p!}\Big(v_1(S\cup\{j\})-v_1(S)\Big) + \sum_{S\subseteq P\setminus\{j\}} \frac{|S|!\,(p-|S|-1)!}{p!}\Big(v_2(S\cup\{j\})-v_2(S)\Big)$$
$$= \phi_{j,v_1} + \phi_{j,v_2}. \qquad \square$$

The symmetry axiom requires a little more calculation.

Theorem (Symmetry). *Let $\{\phi_j\}_{j\in P}$ be the Shapley values induced by v. For any two players $j,k\in P$ it holds: if $v(S\cup\{j\}) = v(S\cup\{k\})$ for all $S\subseteq P\setminus\{j,k\}$, then $\phi_j = \phi_k$.*

Proof. Assume $p\geq 2$ and $j,k\in P$ and $v(S\cup\{j\}) = v(S\cup\{k\})$ for all subsets $S\subseteq P\setminus\{j,k\}$. We need to prove that $\phi_j = \phi_k$.

We will use the set-based definition of Shapley values, and will show the claim by straightforward calculation and rearranging the sum. We first split the sum, which is over all sets $S\subseteq P\setminus\{j\}$, into two parts, depending on whether player k is in S or not:

$$\phi_j = \sum_{S\subseteq P\setminus\{j\}} \frac{|S|!\,(p-|S|-1)!}{p!}\Big(v(S\cup\{j\})-v(S)\Big)$$
$$= \sum_{S\subseteq P\setminus\{j,k\}} \frac{|S|!\,(p-|S|-1)!}{p!}\Big(v(S\cup\{j\})-v(S)\Big) + \sum_{\substack{S\subseteq P\setminus\{j\},\\ k\in S}} \frac{|S|!\,(p-|S|-1)!}{p!}\Big(v(S\cup\{j\})-v(S)\Big).$$

Now, in the first part of the sum we can directly use our assumption:

$$\sum_{S\subseteq P\setminus\{j,k\}} \frac{|S|!\,(p-|S|-1)!}{p!}\Big(v(S\cup\{j\})-v(S)\Big) = \sum_{S\subseteq P\setminus\{j,k\}} \frac{|S|!\,(p-|S|-1)!}{p!}\Big(v(S\cup\{k\})-v(S)\Big)$$

For the second part, first note that both sums have the same number of summands, since there is a one-to-one correspondence (a bijection) between the subsets of $P\setminus\{j,k\}$ and those subsets of $P\setminus\{j\}$ that contain k, by either adding k to a given subset $S\subseteq P\setminus\{j,k\}$ or not. We can use this bijection to define an "index transformation" for the second sum:

We replace the set S, which we know must contain k, with $S = \tilde{S} \cup \{k\}$, in other words we define $\tilde{S} := S \setminus \{k\}$. We can then use our assumption on the set $\tilde{S}$ as well. This yields:

$$\begin{aligned}
&\sum_{\substack{S \subseteq P \setminus \{j\}, \\ k \in S}} \frac{|S|!\,(p - |S| - 1)!}{p!} \Big(v(S \cup \{j\}) - v(S) \Big) \\
&= \sum_{\tilde{S} \subseteq P \setminus \{j,k\}} \frac{(|\tilde{S}| + 1)!\,(p - (|\tilde{S}| + 1) - 1)!}{p!} \Big(v(\tilde{S} \cup \{j,k\}) - v(\tilde{S} \cup \{k\}) \Big) \\
&= \sum_{\tilde{S} \subseteq P \setminus \{j,k\}} \frac{(|\tilde{S}| + 1)!\,(p - |\tilde{S}| - 2)!}{p!} \Big(v(\tilde{S} \cup \{j,k\}) - v(\tilde{S} \cup \{j\}) \Big) \\
&= \sum_{\substack{S \subseteq P \setminus \{k\}, \\ j \in S}} \frac{|S|!\,(p - |S| - 1)!}{p!} \Big(v(S \cup \{k\}) - v(S) \Big).
\end{aligned}$$

In the last step, we used this transformation backwards, so we add / remove j instead of k this time, in other words $S = \tilde{S} \cup \{j\}$ and $\tilde{S} := S \setminus \{j\}$. This is possible since $\tilde{S}$ contains neither j nor k.

We now use the first step from the beginning backwards and arrive at the desired result:

$$\begin{aligned}
\phi_j \;&=\; \sum_{S \subseteq P \setminus \{j,k\}} \frac{|S|!\,(p - |S| - 1)!}{p!} \Big(v(S \cup \{k\}) - v(S) \Big) \;+\; \sum_{\substack{S \subseteq P \setminus \{k\}, \\ j \in S}} \frac{|S|!\,(p - |S| - 1)!}{p!} \Big(v(S \cup \{k\}) - v(S) \Big) \\
&= \sum_{S \subseteq P \setminus \{k\}} \frac{|S|!\,(p - |S| - 1)!}{p!} \Big(v(S \cup \{k\}) - v(S) \Big) \;=\; \phi_k. \qquad \square
\end{aligned}$$

b) **Bonus:** Prove the efficiency axiom for Shapley values.

Hint

This is easier when using the order definition. Plug the definition into the sum inside the axiom, then swap the sums and argue why almost all terms of the inner sum vanish.

Solution

Efficiency requires a little more effort than the others:

Theorem (Efficiency). *Let $\{\phi_j\}_{j \in P}$ be the Shapley values induced by v. Then*

$$\sum_{j=1}^{p} \phi_j = v(P).$$

The proof idea is as follows: Because we sum up the Shapley values over all players, the values $v(S)$ of each coalition not equal to P, so $S \subsetneq P$, appear with both minus and plus signs that exactly cancel each other out. Apart from that, the term $v(P)$ occurs for every player, and one only needs to calculate that its weights sum up to 1.

We will use the order-based definition of Shapley values here, since with it the proof is simpler.

Proof. We first plug in the definition and swap the two sums:

$$\sum_{j=1}^{p} \phi_j \;=\; \sum_{j=1}^{p} \frac{1}{p!} \sum_{\pi \in \mathfrak{S}_P} \Big(v\big(\mathrm{Pred}_\pi(j) \cup \{j\}\big) - v\big(\mathrm{Pred}_\pi(j)\big) \Big) = \frac{1}{p!} \sum_{\pi \in \mathfrak{S}_P} \sum_{j=1}^{p} \Big(v\big(\mathrm{Pred}_\pi(j) \cup \{j\}\big) - v\big(\mathrm{Pred}_\pi(j)\big) \Big).$$

Now the inner sum corresponds to a single permutation, and sums over all players in this permutation.

Fix a particular permutation π. We consider the order of the players given by this permutation: $(\pi(1), \pi(2), \ldots, \pi(p))$. Each marginal contribution inside the inner sum corresponds to one step in this ordering. We reorder the inner sum according to this ordering given by the permutation:

$$\begin{aligned}
\sum_{j=1}^{p} \Big(v\big(\mathrm{Pred}_\pi(j) \cup \{j\}\big) - v\big(\mathrm{Pred}_\pi(j)\big) \Big) &= \sum_{i=1}^{p} \Big(v\big(\{\pi(1), \ldots, \pi(i)\}\big) - v\big(\{\pi(1), \ldots, \pi(i-1)\}\big) \Big) \\
&= v\big(\{\pi(1)\}\big) - v(\varnothing) \;+\; v\big(\{\pi(1), \pi(2)\}\big) - v\big(\{\pi(1)\}\big) \\
&\quad \;+\; \ldots \;+\; v\big(\{\pi(1), \ldots, \pi(p)\}\big) - v\big(\{\pi(1), \ldots, \pi(p-1)\}\big) \\
&= v\big(\{\pi(1), \ldots, \pi(p)\}\big) - v(\varnothing) = v(P).
\end{aligned}$$

In other words, the inner sum is a telescope sum, where all the terms except the first and the last one cancel. Plugging this into the outer sum yields the desired result:

$$\sum_{j=1}^{p} \phi_j \;=\; \frac{1}{p!} \sum_{\pi \in \mathfrak{S}_P} v(P) \;=\; \frac{1}{p!} \cdot \big(p! \cdot v(P)\big) \;=\; v(P). \qquad \square$$

When using the set-based definition, one can show that the term $v(P)$ appears exactly once per player (hence p times in total) for the coalition $S = P \setminus \{j\}$ with the weight $\frac{(p-1)!\,\big(p-(p-1)-1\big)!}{p!} = \frac{1}{p}$ each, and then one just has to show that for all the other terms, their number of appearance together with their weights cause them to cancel out. See also this discussion.

c) **Bonus:** Similarly to the additivity axiom, prove the following homogeneity axiom:

If $\alpha \in \mathbb{R}$ is a real number and v the payout function of a game, and we define $\tilde{v} := \alpha v$ as another payout function, then the Shapley value also scales accordingly, that is for all features $j \in P$ we have:

$$\phi_{j,\tilde{v}} = \phi_{j,\alpha v} = \alpha \phi_{j,v}.$$

Note: This axiom together with the additivity axiom shows that the Shapley value is linear in the payout function, so we have linearity of the Shapley value.

Solution

This is as straightforward as the additivity:

Theorem (Linearity). *Let $\alpha, \beta \in \mathbb{R}$ and $v, v_1, v_2 : 2^P \to \mathbb{R}$ be value functions with $v = \alpha v_1 + \beta v_2$, which means that for every subset $S \subseteq P$ we have $v(S) = \alpha \cdot v_1(S) + \beta \cdot v_2(S)$. Let $\{\phi_{j,v}\}_{j \in P}$ be the Shapley values induced by v and $\{\phi_{j,v_1}\}_{j \in P}$ and $\{\phi_{j,v_2}\}_{j \in P}$ those for v_1 and v_2 respectively. Then for any player $j \in P$ it holds:*

$$\phi_{j,v} = \phi_{j,\alpha v_1 + \beta v_2} = \alpha \phi_{j,v_1} + \beta \phi_{j,v_2}.$$

Proof. Since additivity was already proven in part a), we only have to prove homogeneity, that means that $\phi_{j,\alpha v_1} = \alpha \phi_{j,v_1}$ for any $\alpha \in \mathbb{R}$.

So let $p \geq 1$, $j \in P$, $\alpha \in \mathbb{R}$ and v_1 be any value function. Denote $v = \alpha v_1$. For the marginal contributions of player j on any subset $S \subseteq P$ we have: $v(S \cup \{j\}) - v(S) = \alpha v_1(S \cup \{j\}) - \alpha v_1(S) = \alpha\Big(v_1(S \cup \{j\}) - v_1(S)\Big)$. As for the additivity, plugging this into either of the two definitions yields the result:

$$\begin{aligned}
\phi_{j,v} &= \sum_{S \subseteq P \setminus \{j\}} \frac{|S|!\,(p - |S| - 1)!}{p!} \Big(v(S \cup \{j\}) - v(S)\Big) = \sum_{S \subseteq P \setminus \{j\}} \frac{|S|!\,(p - |S| - 1)!}{p!}\, \alpha\Big(v_1(S \cup \{j\}) - v_1(S)\Big) \\
&= \alpha \sum_{S \subseteq P \setminus \{j\}} \frac{|S|!\,(p - |S| - 1)!}{p!} \Big(v_1(S \cup \{j\}) - v_1(S)\Big) = \alpha \phi_{j,v_1}. \qquad \square
\end{aligned}$$

Exercise 3: SHAP

Learning goals

1. Apply Shapley values as a local feature relevance tool to a machine learning model.
2. Implement the marginal sampling-based SHAP value function and plug it into the Shapley approximation.
3. Implement Kernel SHAP and interpret the resulting SHAP values.

In this exercise, we want to apply Shapley values as a local feature relevance quantification tool to a machine learning model and also use the SHAP framework for this. We will implement both the simple sampling-based approximation algorithm for vanilla Shapley values from the lecture, as well as the Kernel SHAP algorithm.

a) Load the FIFA dataset (containing all matches from the FIFA world cup 2018) from data/fifa.csv into a data table or data frame (e.g., using the pandas library in Python) and train a random forest to predict the target variable 'Man of the Match'. This means that the model predicts the probability of a team having the 'Man of the Match' amongst them.

Hint

Note that many of the float variables in the data set contain missing values, hence you may consider to use integer variables only. Transform the target variable 'Man of the Match' into a binary format suitable for a prediction task and select only explanatory variables of type integer, or only those without missing values. Don't forget to split the data into a training and test set before fitting the random forest! Use an instance from your test set to generate an exemplary prediction for the 'Man of the Match' probability.

Solution

We read the data, binarize the target, keep only complete numeric columns, split into training and test set and fit a random forest:

R

```r
### Read the dataset via the read.csv()-function.
df = read.csv(file = "../data/fifa.csv")

### Transform the character variable Man.of.the.Match ("Yes"/"No") into a
### logical variable. This has to happen before deleting all character
### variables, else we would remove our target variable as well.
df$Man.of.the.Match = ifelse(df$Man.of.the.Match == "Yes", TRUE, FALSE)

### Remove all variables of type character.
df = df[, !sapply(df, is.character)]

### Delete all columns containing at least one NA value.
df = df[, apply(df, 2, function(x) !any(is.na(x)))]

### Classical training and testing split.
train_idx = sample(nrow(df), 0.7 * nrow(df))
training_data = df[train_idx, ]
test_data = df[-train_idx, ]

### Split into features and target, which will be used later on.
X_train = training_data[, -which(names(training_data) == "Man.of.the.Match")]
X_test = test_data[, -which(names(test_data) == "Man.of.the.Match")]
y_train = training_data$Man.of.the.Match
y_test = test_data$Man.of.the.Match

### Model a random forest using the ranger package (others are possible too).
classifier_RF = ranger(Man.of.the.Match ~ ., training_data)
print(classifier_RF)

### Exemplary predictions for instances from the test set.
predict(classifier_RF, X_test[1:5, ])$predictions
```

```
Ranger result

Call:
 ranger(Man.of.the.Match ~ ., training_data)

Type:                             Classification
Number of trees:                  500
Sample size:                      89
Number of independent variables:  18
Mtry:                             4
Target node size:                 1
Variable importance mode:         none
```

```
Splitrule:                      gini
OOB prediction error:           24.72 %
```

1. 1 2. 1 3. 1 4. 1 5. 1

Python

```
df = pd.read_csv('../data/fifa.csv')
X, y = df.loc[:, df.columns != 'Man of the Match'], df['Man of the Match']

# Binarize the target and keep only complete numeric columns.
y = (y == 'Yes')
X = X.select_dtypes([np.number]).copy()
X = X.dropna(axis=1)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

classifier_RF = RandomForestClassifier()
classifier_RF.fit(X_train, y_train)

# For Shapley values we work with the predicted probability of class 'Yes'.
predict_RF = lambda x: classifier_RF.predict_proba(x)[:, 1]

# Exemplary predictions for instances from the test set.
print(X_test.shape)
print(classifier_RF.predict(X_test.iloc[1:5]))
print(predict_RF(X_test.iloc[1:5]))
```

```
(39, 18)
[False False  True  True]
[0.11 0.3  0.55 0.77]
```

As a classifier, this model of course produces probabilities as outputs, which are then transformed into a binary prediction. This is not very well suited for Shapley values, since they basically require a numerical or continuous target, in particular for the efficiency axiom to hold up. We therefore work with the predicted probabilities where possible; since the Shapley or SHAP values are obtained through averaging, they then still always represent probabilities or relative frequencies.

b) From the different possibilities to choose the value function $g(\mathbf{z}')$ in the SHAP framework, here we use the marginal sampling-based SHAP value function $g(\mathbf{z}') = \mathbb{E}_{\mathbf{X}_{-S}}[\hat{f}(\mathbf{x}_S, \mathbf{X}_{-S})]$, so we use PD-functions. Implement the function `marginal_vfunc(S, observation, X, predict, nr_samples)`, which approximates the PD-function value for a feature subset `S`, evaluated at the data point `observation`, via sampling background data from the

data set `X`; in other words, implement a Monte-Carlo approximation of this PD-function evaluated at point `observation`. `predict` is a function that returns the model predictions for a single data point or a set of data points.

Test your implementation on different feature subsets for the instance from a).

Solution

Pseudocode of `marginal_vfunc(S, observation, X, predict, nr_samples)`:

```
Require S: feature index or indices, or feature names
Require observation: observation where to evaluate the marginal expectation
Require X: feature matrix
Require predict: function returning the predictions of the model
Require nr_samples: number of samples
  X_tmp <- sample nr_samples samples from X (with replacement)
  X_tmp <- replace features S with respective values from observation
  return mean of predict(X_tmp)
```

R

```r
### The marginal sampling-based SHAP value function, a Monte-Carlo
### approximation of the PD-function evaluated at `observation`.
marginal_vfunc = function(S, observation, X, model, nr_samples = 100) {
  ### Get nr_samples random observations from the given dataset.
  X_tmp = X[sample(nrow(X), nr_samples, replace = TRUE), ]

  ### Replace the features given in S by the values of the observation.
  X_tmp[, S] = observation[, S]

  ### Predict and average.
  mean(predict(model, X_tmp)$predictions)
}

### Testing on different feature subsets.
marginal_vfunc(names(X_train)[1:3], X_test[1, ], X_train, classifier_RF, nr_samples = 1000)
marginal_vfunc(names(X_train)[1], X_test[1, ], X_train, classifier_RF, nr_samples = 1000)
marginal_vfunc(names(X_train)[1:2], X_test[1, ], X_train, classifier_RF, nr_samples = 1000)
marginal_vfunc(names(X_train)[1:2], X_test[2, ], X_train, classifier_RF, nr_samples = 1000)
```

0.515

0.541

0.525

0.651

Python

```
def marginal_vfunc(S, observation, X, predict, nr_samples=100):
    """Monte-Carlo approximation of the PD-function at `observation`."""
    X_tmp = X.sample(nr_samples, replace=True).copy()
    # Reshape and copy the observation nr_samples many times.
    observation_df = pd.DataFrame(observation).T.sample(nr_samples, replace=True)
    observation_df = observation_df.reset_index(drop=True)
    X_tmp.loc[:, S] = np.array(observation_df.loc[:, S].copy())
    return np.mean(predict(X_tmp))


# Testing on different feature subsets.
print(marginal_vfunc(list(X_test.columns[0:3]), X_test.iloc[1, ], X_train, predict_RF, nr_samples=10
print(marginal_vfunc(list(X_test.columns[0:1]), X_test.iloc[1, ], X_train, predict_RF, nr_samples=10
print(marginal_vfunc(list(X_test.columns[0:2]), X_test.iloc[1, ], X_train, predict_RF, nr_samples=10
print(marginal_vfunc(list(X_test.columns[0:2]), X_test.iloc[2, ], X_train, predict_RF, nr_samples=10
```

```
0.23195999999999997
0.28779999999999994
0.27222
0.5689099999999999
```

c) By using this value function `marginal_vfunc()` inside our Shapley value implementations from Exercise 1, we can approximate the Shapley values for our model. This is almost the approximation algorithm for Shapley values discussed in the lecture.

Calculate `shapley_perm_approx()` from the Shapley value implementations using `marginal_vfunc()` and the random forest from a) on your test instance.

Solution

We plug `marginal_vfunc()` in as the value function when calling `shapley_perm_approx()` and need to take care that the other arguments of `marginal_vfunc()` are handled correctly.

R

```
### Wrap the value function so that it only depends on the coalition; the
### Shapley implementations from Exercise 1 can then be reused directly.
model_specific_marginal_vfunc = function(coalition) {
  marginal_vfunc(coalition, X_test[1, ], X_train, classifier_RF, nr_samples = 100)
}

print(shapley_perm_approx(names(X_test)[3], names(X_test),
  v_function = model_specific_marginal_vfunc))
shapley_test(names(X_test), shapley_perm_approx, v_function = model_specific_marginal_vfunc)
```

```
[1] -0.018
 [1]  0.1223 -0.0057 -0.0263 -0.0125  0.0437  0.0499 -0.0648  0.1787  0.0900
[10] -0.0185 -0.0304  0.1344  0.1050 -0.0134  0.0021  0.0019  0.0019  0.0023
[1] 0.5606
[1] 1
[1] FALSE
```

Python

```
# Plug marginal_vfunc into the Shapley implementations from Exercise 1; the
# extra arguments of marginal_vfunc are passed through via *args.
# NOTE: This takes a while.
print(shapley_perm_approx(X_test.columns[2], list(X_test.columns),
                          marginal_vfunc, X_test.iloc[0, ], X_train, predict_RF))
shapley_test(list(X_test.columns), shapley_perm_approx,
             marginal_vfunc, X_test.iloc[0, ], X_train, predict_RF)
```

```
-0.05654700000000001
Player Goal Scored: 0.04934100000000001
Player Ball Possession %: -0.05565799999999999
Player Attempts: -0.052247999999999975
Player On-Target: -0.0018279999999999941
Player Off-Target: -0.015306999999999982
Player Blocked: -0.013499
Player Corners: -0.014595999999999968
Player Offsides: 0.017806000000000016
Player Free Kicks: -0.014251999999999985
Player Saves: -0.029467999999999987
```

```
Player Pass Accuracy %: 0.026905000000000016
Player Passes: -0.03845399999999999
Player Distance Covered (Kms): 0.016701000000000018
Player Fouls Committed: -0.00407899999999998
Player Yellow Card: 0.02395599999999999
Player Yellow & Red: 0.0019250000000000029
Player Red: 0.00159899999999999
Player Goals in PSO: -6.999999999981465e-06
-0.10116299999999978
0.4200000000000001
False
```

Here we see exactly the problem mentioned in a): the (approximated) Shapley values represent probabilities, whereas a single prediction is binary, so efficiency does not need to hold exactly in this situation.

d) The SHAP framework, which is based on locally fitting a weighted linear model, can be used to calculate SHAP values. Besides the vanilla framework, which basically uses the same algorithm as the one we implemented so far, other more efficient equivalent algorithms for calculating SHAP values have been proposed, among them Kernel SHAP. Your task is to implement Kernel SHAP and calculate the SHAP values for your test instance. Do this by writing the following functions:

- `shap_weights(mask)`: A function to return the weights for a given `mask` containing the sampled coalitions, so `mask` is either a single coalition vector or a matrix containing K many coalition vectors.
- `replace_dataset(obs, X, nr_samples)`: A function to create the dataset of `nr_samples` many samples generated in the first and second step of the Kernel SHAP calculations. This includes generating the coalitions, saving them in `mask`, and mapping the binary feature space back to the original feature space. You can directly sample `nr_samples` many data points as well as coalitions, so sampling a new random background sample for each coalition, you do not need to generate multiple background samples for every coalition or use the same background sample for different coalitions. The positions in the mask indicate where to replace the random values with the ones of the given observation `obs`.
- `shap_data(obs, X, nr_samples, predict)`: A function summarizing `replace_dataset()`, `predict()`, and `shap_weights()`. As before, `predict` is a function that returns the model predictions for a single data point or a set of data points.
- `kernel_shap(obs, X, nr_samples, predict)`: A function implementing the complete sampling-based Kernel SHAP function.

Solution

`shap_weights()` implements the Shapley kernel $\pi_x(|z'|) = \frac{p-1}{\binom{p}{|z'|}\,|z'|\,(p-|z'|)}$ where $|z'|$ is the coalition size; `replace_dataset()` samples background data and overwrites the masked positions with the observation's values; `kernel_shap()` fits the weighted linear model on the binary masks and returns its coefficients.

R

```
### Sample for every entry in our data whether to include it or not.
sample_mask = function(nrow, ncol) {
  matrix(rbinom(nrow * ncol, 1, 0.5), nrow = nrow)
}

### The Shapley kernel weights pi_x(|z'|), where |z'| is the number of
### included features (given by a 1 in the mask).
shap_weights = function(mask) {
  p = ncol(mask)
  z_prime_abs = rowSums(mask)
  (p - 1) / (choose(p, z_prime_abs) * z_prime_abs * (p - z_prime_abs))
}

### Create the dataset generated during the SHAP calculations: nr_samples
### background samples, where the positions with mask == 1 are replaced by
### the values of the given observation obs.
replace_dataset = function(obs, X, n_samples, mask) {
  X_new = X[sample(nrow(X), n_samples, replace = TRUE), ]
  obs_rep = obs[rep(1, n_samples), ]

  ### At positions where the mask is 1 we effectively replace the original
  ### entries by the ones of the observation.
  X_sample = X_new - mask * (X_new - obs_rep)
  X_sample
}

### Putting things together: the sampling-based Kernel SHAP function.
kernel_shap = function(obs, X, n_samples, model) {
  mask = sample_mask(n_samples, ncol(X))
  df = replace_dataset(obs, X, n_samples, mask)
  weights = shap_weights(mask)
  pred = predict(model, df)$predictions

  ### Save the mask as well as the respective predictions.
  df_shap = as.data.frame(mask)
  names(df_shap) = names(X)
  df_shap$pred = pred

  ### Fit a weighted linear model using the above calculated weights. Since
  ### we are in a binary setting, we perform logistic regression.
  wls_model = suppressWarnings(
    glm(pred ~ ., family = binomial, data = df_shap, weights = weights))
  wls_model$coefficients
}
```

Python

```python
def shap_weights(mask):
    """Shapley kernel weights pi_x(|z'|), |z'| = coalition size per row."""
    p = mask.shape[1]
    zs = np.sum(mask, axis=1)
    nominator = (p - 1)
    denominator = comb(p, zs) * zs * (p - zs)
    return nominator / denominator


def replace_dataset(obs, X, nr_samples):
    """Sample background data and coalitions; replace masked positions with
    the observation's values."""
    X_new = np.array(X.sample(nr_samples, replace=True))
    obs_rep = np.repeat(np.array(obs).reshape(-1, 1), X_new.shape[0], axis=1).T
    mask = np.random.binomial(1, 0.5, np.prod(X_new.shape)).reshape(X_new.shape)
    row_ixs, col_ixs = np.where(mask)
    X_new[row_ixs, col_ixs] = obs_rep[row_ixs, col_ixs]
    X_new = pd.DataFrame(X_new, columns=X.columns)
    return X_new, mask


def shap_data(obs, X, nr_samples, predict):
    X_new, mask = replace_dataset(obs, X, nr_samples)
    weight = shap_weights(mask)
    pred = predict(X_new)
    return mask, pred, weight


def kernel_shap(obs, X, nr_samples, predict):
    """The complete sampling-based Kernel SHAP function: fit a weighted
    linear model on the binary masks and return its coefficients."""
    mask, pred, weight = shap_data(obs, X, nr_samples, predict)
    lm = LinearRegression()
    lm.fit(mask, pred, weight)
    coef = pd.Series(lm.coef_, index=X.columns)
    coef['intercept'] = lm.intercept_
    return coef
```

e) Calculate `kernel_shap(X_test[1, ], X_train, 1000, classifier_RF)` using the random forest from a) and interpret the resulting SHAP values in this context.

Solution

R

```
### Calculate the KernelSHAP values for the first instance of the test set.
obs = X_test[1, ]
shap_vals = kernel_shap(obs, X_train, 1000, classifier_RF)
shap_vals
```

(Intercept) -0.514332829764812 **Goal.Scored** 0.952961395011039 **Ball.Possession..** -0.241785565507813 **Attempts** -1.13861019484497 **On.Target** 0.22125218074014 **Off.Target** 0.932887018950052 **Blocked** 0.377587082441724 **Corners** -0.893633068140951 **Offsides** 0.705409565676433 **Free.Kicks** 0.487929458064504 **Saves** -0.560219491826692 **Pass.Accuracy..** 0.0713167115276342 **Passes** 0.871915257655272 **Distance.Covered..Kms.** 0.671440995863163 **Fouls.Committed** -0.272718408083098 **Yellow.Card** 0.0994128843591609 **Yellow...Red** -0.0889844301835035 **Red** 0.209466643581212 **Goals.in.PSO** -0.365104441738126

Python

```
# Note: Here we predict probabilities, and a standard linear regression is
# used, not logistic regression.
shap_vals = kernel_shap(X_test.iloc[0, :], X_train, 1000, predict_RF)
print(shap_vals)
```

```
Goal Scored              -0.016307
Ball Possession %        -0.028468
Attempts                 -0.051682
On-Target                -0.001449
Off-Target               -0.002331
Blocked                  -0.015557
Corners                   0.005354
Offsides                  0.016914
Free Kicks                0.001798
Saves                    -0.022184
Pass Accuracy %           0.001223
Passes                   -0.022031
Distance Covered (Kms)    0.033001
Fouls Committed           0.004705
```

```
Yellow Card              0.035847
Yellow & Red            -0.012275
Red                      0.053806
Goals in PSO             0.015558
intercept                0.453397
dtype: float64
```

The coefficients of the weighted linear model are the SHAP values of the features for this test instance: features with positive values push the predicted 'Man of the Match' probability of this observation up, those with negative values pull it down, relative to the average prediction (the intercept). The specific results and their accuracy depend on the implementation, the seed and the number of samples.

Exercise 4: LIME

Learning goals

1. Apply an existing LIME implementation to a fitted model.
2. Explain the influence of hyperparameters such as the kernel width on LIME explanations.
3. Compare LIME and SHAP conceptually.

LIME is a local feature effect method that is based on locally approximating the decision function using a linear model. Then, the linear model coefficients are interpreted as usual. Like SHAP, LIME is an additive attribution technique.

a) Apply a LIME implementation to the dataset from Exercise 3.

Hint

Implementations are available in Python and R.

Solution

R

We use the `DALEX`/`DALEXtra` interface, which wraps the `lime` package:

```
### Fit a random forest with the randomForest package (the DALEX LIME
### interface expects a classification model here).
rf_model = randomForest(as.factor(Man.of.the.Match) ~ ., training_data)

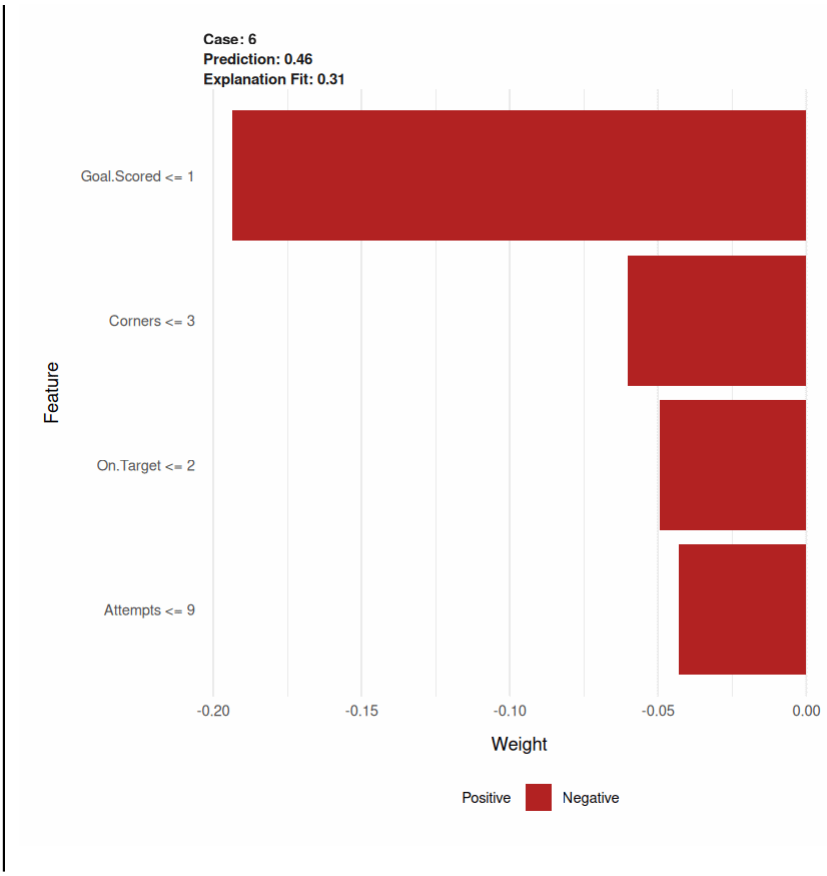
model_ex = DALEX::explain(model = rf_model,
  data = X_train,
  y = as.numeric(y_train),
  label = "Random Forest")

### Local explanations for the model via LIME.
model_type.dalex_explainer = DALEXtra::model_type.dalex_explainer
predict_model.dalex_explainer = DALEXtra::predict_model.dalex_explainer

iml_obs = predict_surrogate(explainer = model_ex,
  new_observation = X_test[1, ],
  type = "lime")

plot(iml_obs)
```

```
Preparation of a new explainer is initiated
  -> model label       :  Random Forest
  -> data              :  89  rows  18  cols
  -> target variable   :  89  values
  -> predict function  :  yhat.randomForest  will be used (  default  )
  -> predicted values  :  No value for predict function target column. (  default  )
  -> model_info        :  package randomForest , ver. 4.7.1.2 , task classification (  d
  -> predicted values  :  numerical, min =  0.028 , mean =  0.4683146 , max =  0.938
  -> residual function :  difference between y and yhat (  default  )
  -> residuals         :  numerical, min =  -0.346 , mean =  0.003595506 , max =  0.318
  A new explainer has been created!
```

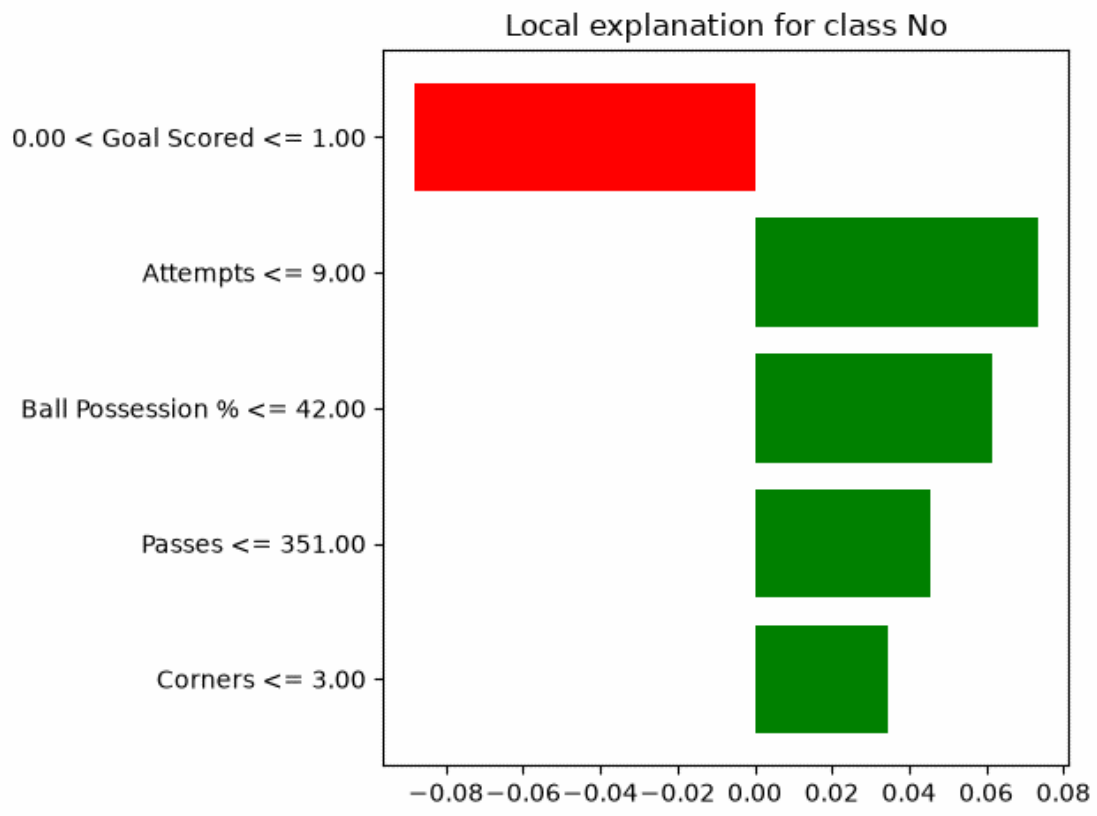

Case: 6
Prediction: 0.46
Explanation Fit: 0.31
Goal.Scored <= 1
Corners <= 3
On.Target <= 2
Attempts <= 9
Feature
-0.20
-0.15
-0.10
-0.05
0.00
Weight
Positive
Negative

Python

We use the `lime` package directly:

```
explainer = lime.lime_tabular.LimeTabularExplainer(
    np.array(X_train), feature_names=X.columns,
    class_names=['No', 'Yes'])

exp = explainer.explain_instance(np.array(X_test.iloc[0, :]), classifier_RF.predict_proba,
                                 num_features=5, top_labels=1)
exp.as_pyplot_figure(label=exp.available_labels()[0])
plt.tight_layout()
plt.show()
```



b) Try different hyperparameter configurations (for instance, modify the kernel width). How does that influence the result?

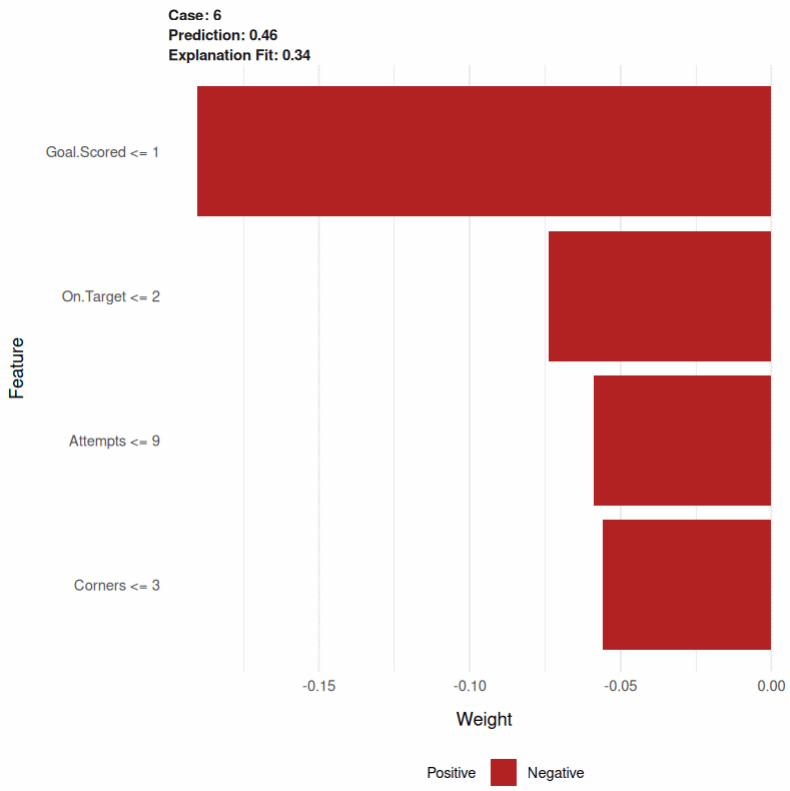
Solution

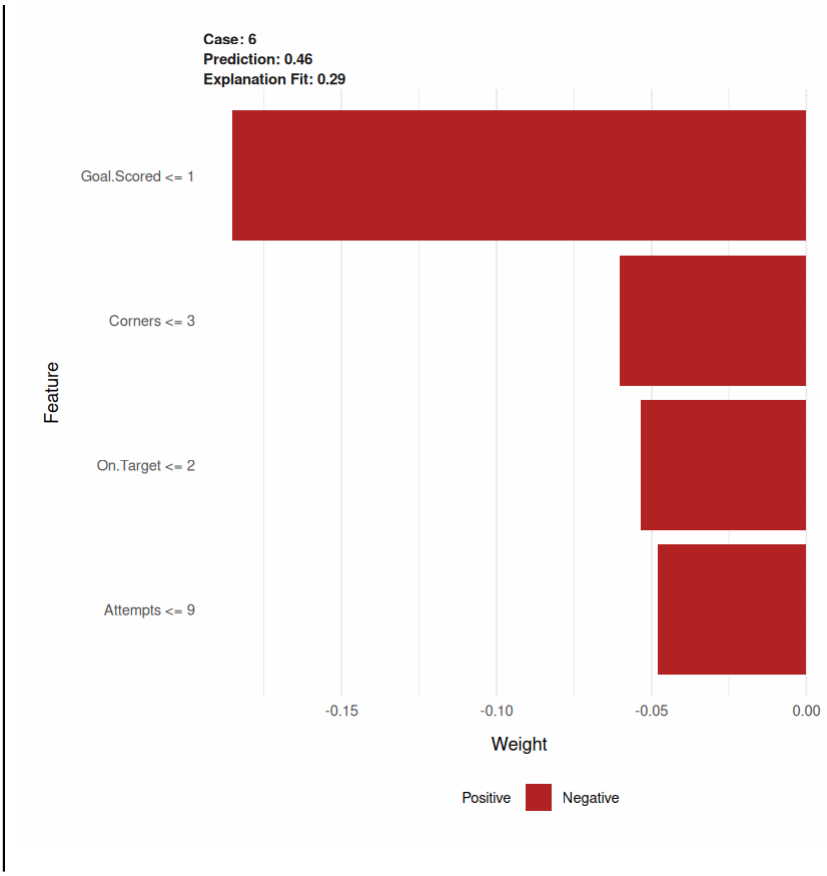
The kernel width controls the size of the neighborhood: a large kernel width weights distant samples almost as much as close ones, so the explanation approaches a global surrogate; a very small kernel width makes the explanation extremely local and unstable.

R

```
### Set the kernel width to 10.
iml_obs = predict_surrogate(explainer = model_ex,
  new_observation = X_test[1, ],
  type = "lime",
  kernel_width = 10)
plot(iml_obs)

### Set the kernel width to 0.001.
iml_obs = predict_surrogate(explainer = model_ex,
  new_observation = X_test[1, ],
  type = "lime",
  kernel_width = 0.001)
plot(iml_obs)
```

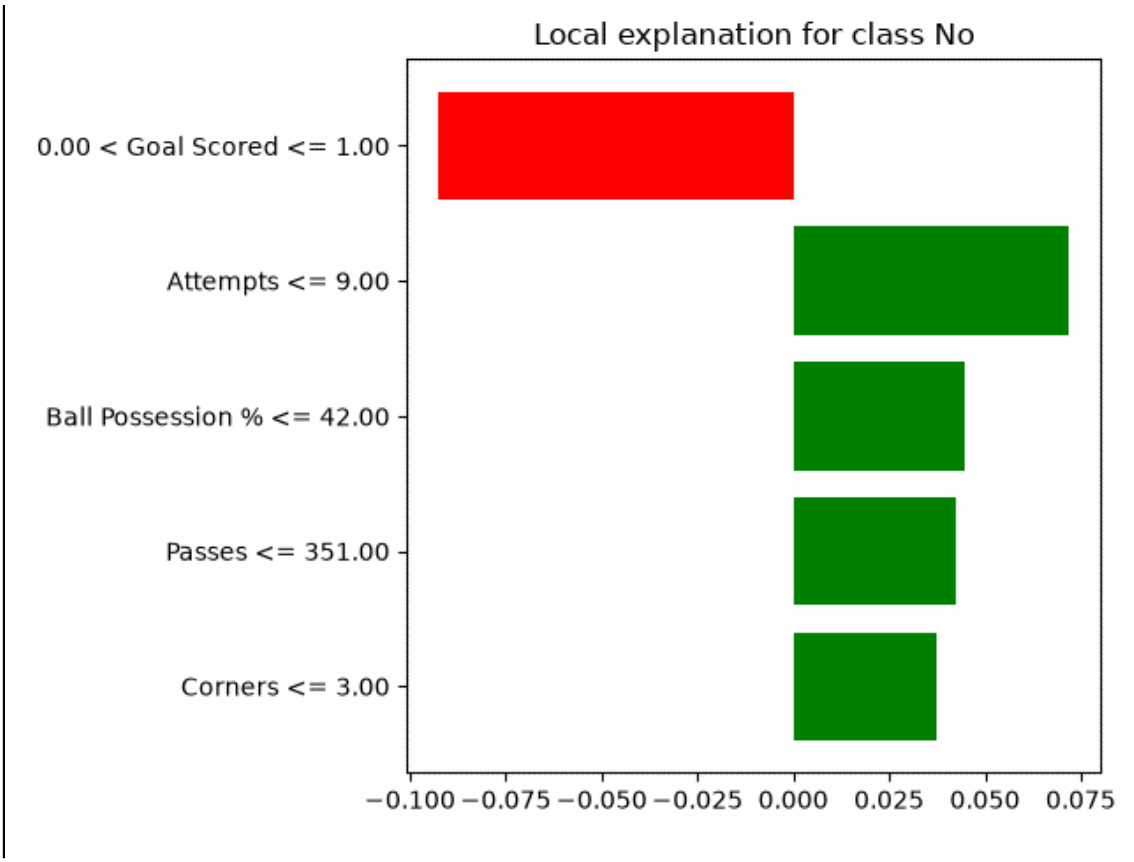


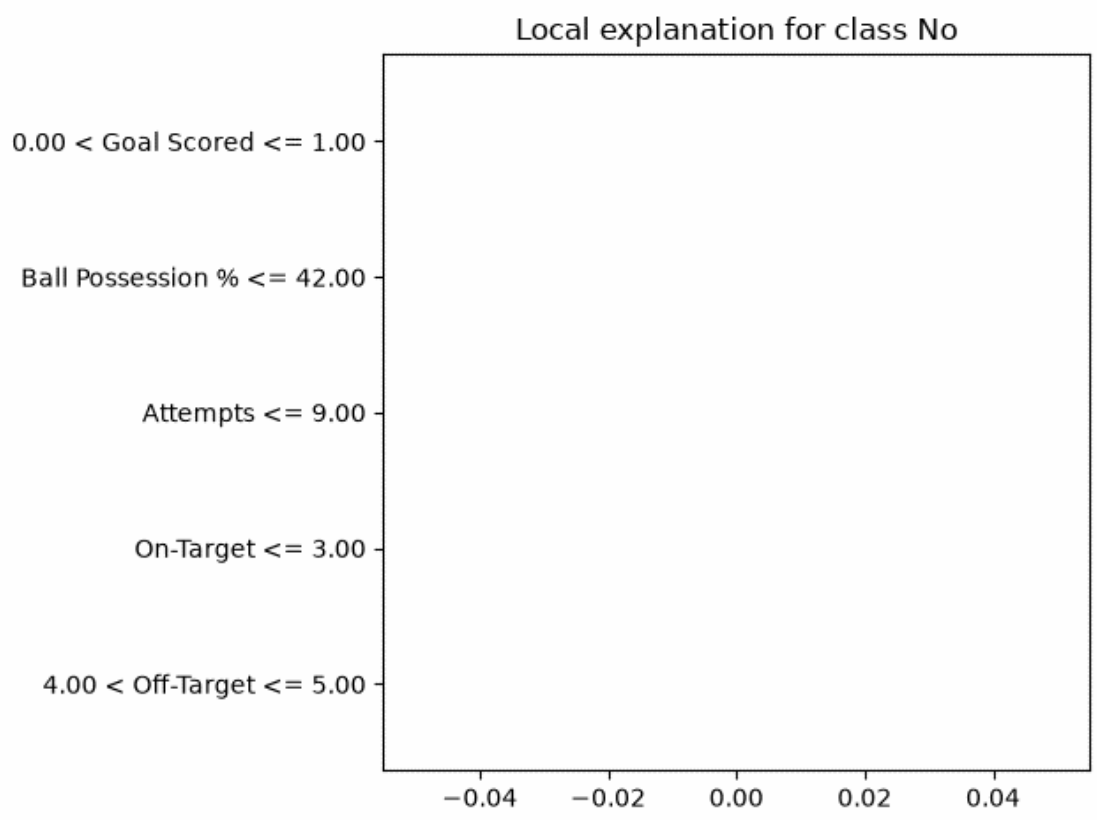

Case: 6
Prediction: 0.46
Explanation Fit: 0.29
Goal.Scored <= 1
Corners <= 3
On.Target <= 2
Attempts <= 9
Feature
-0.15
-0.10
-0.05
0.00
Weight
Positive
Negative

Python

```
# Large kernel width: the neighborhood is wide, the explanation approaches a
# global surrogate.
explainer_wide = lime.lime_tabular.LimeTabularExplainer(
    np.array(X_train), feature_names=X.columns,
    class_names=['No', 'Yes'], kernel_width=10)
exp = explainer_wide.explain_instance(np.array(X_test.iloc[0, :]), classifier_RF.predict_proba,
                                      num_features=5, top_labels=1)
exp.as_pyplot_figure(label=exp.available_labels()[0])
plt.tight_layout()
plt.show()

# Very small kernel width: the explanation is extremely local and unstable.
explainer_narrow = lime.lime_tabular.LimeTabularExplainer(
    np.array(X_train), feature_names=X.columns,
    class_names=['No', 'Yes'], kernel_width=0.001)
exp = explainer_narrow.explain_instance(np.array(X_test.iloc[0, :]), classifier_RF.predict_proba,
                                        num_features=5, top_labels=1)
exp.as_pyplot_figure(label=exp.available_labels()[0])
plt.tight_layout()
plt.show()
```


Local explanation for class No
0.00 < Goal Scored <= 1.00
Attempts <= 9.00
Ball Possession % <= 42.00
Passes <= 351.00
Corners <= 3.00
−0.100
−0.075
−0.050
−0.025
0.000
0.025
0.050
0.075



The results depend on the metric used to define the neighborhood.

c) Compare the result to SHAP. In what way are LIME and SHAP similar? How do the methods differ?

Solution

Both SHAP and LIME rely on a linear model approximation of the model. For LIME, the normal feature values are used, for SHAP a transformed distribution indicating coalition membership for a sample is relied upon. LIME weights the different samples according to their distance to the observation of interest. In contrast, SHAP weights them according to the Shapley kernel weights (which simulate sampling random permutations).